PRAKTIKUM KECERDASAN BUATAN
PERTEMUAN 3

Nama: Aqsha Ramadhan Dimas Haryanto
NIM: H1D024089
Shift Lama: B
Shift Baru: D

Studi Kasus 1

```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib.pyplot as plt
```

Penjelasan: Pada bagian pertama, program mengimpor pustaka (*library*) yang dibutuhkan untuk menjalankan komputasi logika fuzzy di Python, yaitu numpy untuk membuat rentang angka numerik yang menjadi batas nilai setiap variabel, skfuzzy beserta modul control sebagai mesin utama untuk merakit sistem aturan fuzzy, dan matplotlib.pyplot yang bertugas untuk menggambar serta memunculkan antarmuka grafis berupa kurva ke layar komputermu

```
barang_terjual = ctrl.Antecedent(np.arange(0, 101, 1), 'barang_terjual')
permintaan = ctrl.Antecedent(np.arange(0, 301, 1), 'permintaan')
harga_item = ctrl.Antecedent(np.arange(0, 100001, 1), 'harga_item')
profit = ctrl.Antecedent(np.arange(0, 4000001, 1), 'profit')
stok_makanan = ctrl.Consequent(np.arange(0, 1001, 1), 'stok_makanan')
```

Penjelasan: Pada tahap kedua, program mendeklarasikan variabel *input* (Antecedent) dan *output* (Consequent) beserta batas rentang nilainya menggunakan bantuan fungsi np.arange(), di mana didefinisikan empat variabel masukan yaitu barang terjual, permintaan, harga item, dan profit, serta satu variabel keluaran yaitu stok makanan yang dibatasi rentang nilainya dari 0 hingga 1000 agar sesuai dengan semesta pembicaraan pada studi kasus

```
barang_terjual['rendah'] = fuzz.trimf(barang_terjual.universe, [0, 0, 40])
barang_terjual['sedang'] = fuzz.trimf(barang_terjual.universe, [30, 50, 70])
barang_terjual['tinggi'] = fuzz.trimf(barang_terjual.universe, [60, 100, 100])

permintaan['rendah'] = fuzz.trimf(permintaan.universe, [0, 0, 100])
permintaan['sedang'] = fuzz.trimf(permintaan.universe, [50, 150, 250])
permintaan['tinggi'] = fuzz.trimf(permintaan.universe, [0, 200, 300])

harga_item['murah'] = fuzz.trimf(harga_item.universe, [0, 0, 40000])
harga_item['sedang'] = fuzz.trimf(harga_item.universe, [30000, 50000, 100000])
harga_item['mahal'] = fuzz.trimf(harga_item.universe, [50000, 100000, 100000])

profit['rendah'] = fuzz.trimf(profit.universe, [0, 0, 1000000])
profit['sedang'] = fuzz.trimf(profit.universe, [1000000, 2000000, 2500000])
profit['banyak'] = fuzz.trapmf(profit.universe, [1500000, 2500000, 4000000, 4000000])

stok_makanan['sedang'] = fuzz.trimf(stok_makanan.universe, [0, 500, 900])
stok_makanan['banyak'] = fuzz.trimf(stok_makanan.universe, [600, 1000, 1000])
```

Penjelasan: Langkah ketiga adalah mendefinisikan fungsi keanggotaan untuk memetakan nilai-nilai angka ke dalam kategori linguistik seperti rendah, sedang, atau tinggi, dengan memanfaatkan bentuk kurva segitiga (fuzz.trimf) yang digambar dari tiga titik potong untuk sebagian besar himpunan, serta

kurva trapesium (fuzz.trapmf) yang digambar dari empat titik potong khusus untuk himpunan profit dengan kategori 'banyak

```
barang_terjual.view()
permintaan.view()
harga_item.view()
profit.view()
stok_makanan.view()
```

Penjelasan: Bagian keempat ini berfokus pada kebutuhan visualisasi data mentah, di mana program secara berurutan memanggil fungsi .view() pada masing-masing variabel untuk merender gambar kurvanya berdasarkan titik-titik fungsi keanggotaan yang telah dibuat di langkah sebelumnya, yang kemudian dieksekusi secara nyata ke layar melalui pemanggilan perintah plt.show()

```
rule1 = ctrl.Rule(barang_terjual['tinggi'] & permintaan['tinggi'] & harga_item['murah'] & profit['banyak'], stok_makanan['banyak'])
rule2 = ctrl.Rule(barang_terjual['tinggi'] & permintaan['tinggi'] & harga_item['murah'] & profit['sedang'], stok_makanan['sedang'])
rule3 = ctrl.Rule(barang_terjual['tinggi'] & permintaan['sedang'] & harga_item['murah'] & profit['sedang'], stok_makanan['sedang'])
rule4 = ctrl.Rule(barang_terjual['sedang'] & permintaan['tinggi'] & harga_item['murah'] & profit['sedang'], stok_makanan['sedang'])
rule5 = ctrl.Rule(barang_terjual['sedang'] & permintaan['tinggi'] & harga_item['murah'] & profit['banyak'], stok_makanan['banyak'])
rule6 = ctrl.Rule(barang_terjual['rendah'] & permintaan['rendah'] & harga_item['sedang'] & profit['sedang'], stok_makanan['sedang'])
```

Penjelasan:Pada tahap kelima, program menerjemahkan enam aturan logika bersyarat (IF-THEN) dari studi kasus Udin Pet Shop ke dalam sintaks kode menggunakan perintah ctrl.Rule, di mana setiap aturannya secara spesifik menghubungkan kombinasi kondisi dari variabel-variabel input menggunakan operator logika AND (disimbolkan dengan &) untuk menyimpulkan sebuah keputusan pada variabel output stok makanan

```
stok_ctrl = ctrl.ControlSystem([rule1, rule2, rule3, rule4, rule5, rule6])
stok_sim = ctrl.ControlSystemSimulation(stok_ctrl)

# 5. Memasukkan Nilai Input & Eksekusi
stok_sim.input['barang_terjual'] = 80
stok_sim.input['permintaan'] = 255
stok_sim.input['harga_item'] = 25000
stok_sim.input['profit'] = 3500000
```

Penjelasan: Bagian keenam merupakan proses perakitan inti dari program ini, di mana keenam aturan yang telah dijabarkan tadi dibungkus dan digabungkan menjadi satu kesatuan di dalam sebuah ControlSystem, yang kemudian sistem tersebut segera dikonversi menjadi objek simulasi melalui ControlSystemSimulation agar mesin inferensi siap menerima dan mengeksekusi angka masukan yang nyata

```
hasil_stok = stok_sim.output['stok_makanan']
print("--- HASIL PERHITUNGAN LOGIKA FUZZY ---")
print(f"Barang Terjual : 80")
print(f"Permintaan     : 255")
print(f"Harga per Item : 25.000")
print(f"Profit         : 3.500.000")
print("----------------------------------------")
print(f"Jumlah Stok Makanan yang Direkomendasikan: {hasil_stok:.2f} unit")
```

Penjelasan: Pada tahap ketujuh, nilai angka berdasarkan studi kasus secara aktual dimasukkan ke dalam objek simulasi—yaitu barang terjual sebanyak 80, permintaan 255, harga 25.000, dan profit 3.500.000—yang selanjutnya program akan menjalankan perintah .compute() untuk mengalkulasi dan mencocokkan input tersebut dengan seluruh aturan yang ada, hingga akhirnya melewati proses defuzzifikasi untuk menghasilkan sebuah angka rekomendasi final yang dicetak ke layar.

```
plt.xticks(np.arange(0, 101, 10))
stok_makanan.view(sim=stok_sim)
plt.show()
```

Terakhir, pada bagian kedelapan, program sedikit merapikan tampilan jarak angka pada sumbu horizontal grafiknya menggunakan plt.xticks, lalu kembali memanggil fungsi .view() pada variabel stok makanan, namun kali ini dengan menyisipkan argumen simulasi sehingga kurva yang muncul tidak lagi kosong, melainkan sudah dihiasi dengan area arsiran tebal dan sebuah garis lurus yang memvisualisasikan titik pasti dari hasil perhitungan logika fuzzy tersebut.

Studi Kasus 2

```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib.pyplot as plt
```

Penjelasan: Pada bagian pertama, program mengimpor pustaka yang diperlukan untuk memproses logika fuzzy di Python, yaitu numpy untuk mendefinisikan deret angka yang menjadi batas rentang nilai, skfuzzy beserta modul control untuk membangun dan menyimulasikan sistem aturan logika fuzzy, serta matplotlib.pyplot yang bertugas untuk merender dan menampilkan antarmuka grafis berupa kurva ke layar monitormu.

```
kejelasan_informasi = ctrl.Antecedent(np.arange(0, 101, 1), 'kejelasan_informasi')
kejelasan_prasyarat = ctrl.Antecedent(np.arange(0, 101, 1), 'kejelasan_prasyarat')
kemampuan_petugas = ctrl.Antecedent(np.arange(0, 101, 1), 'kemampuan_petugas')
ketersediaan_sarpras = ctrl.Antecedent(np.arange(0, 101, 1), 'ketersediaan_sarpras')

kepuasan_pelayanan = ctrl.Consequent(np.arange(0, 401, 1), 'kepuasan_pelayanan')
```

Penjelasan: Pada tahap kedua, program mendeklarasikan empat variabel masukan (*Antecedent*) yang meliputi kejelasan informasi, kejelasan prasyarat, kemampuan petugas, dan ketersediaan sarpras dengan rentang nilai masing-masing dari 0 hingga 100, serta mendefinisikan satu variabel keluaran (*Consequent*) yaitu kepuasan pelayanan yang memiliki rentang nilai skoring lebih luas yakni dari 0 hingga 400.

```
kejelasan_informasi['Tidak Memuaskan'] = fuzz.trimf(kejelasan_informasi.universe, [0, 0, 75])
kejelasan_informasi['Cukup Memuaskan'] = fuzz.trimf(kejelasan_informasi.universe, [60, 75, 90])
kejelasan_informasi['Memuaskan'] = fuzz.trimf(kejelasan_informasi.universe, [75, 100, 100])

kejelasan_prasyarat['Tidak Memuaskan'] = fuzz.trimf(kejelasan_prasyarat.universe, [0, 0, 75])
kejelasan_prasyarat['Cukup Memuaskan'] = fuzz.trimf(kejelasan_prasyarat.universe, [60, 75, 90])
kejelasan_prasyarat['Memuaskan'] = fuzz.trimf(kejelasan_prasyarat.universe, [75, 100, 100])

kemampuan_petugas['Tidak Memuaskan'] = fuzz.trimf(kemampuan_petugas.universe, [0, 0, 75])
kemampuan_petugas['Cukup Memuaskan'] = fuzz.trimf(kemampuan_petugas.universe, [60, 75, 90])
kemampuan_petugas['Memuaskan'] = fuzz.trimf(kemampuan_petugas.universe, [75, 100, 100])

ketersediaan_sarpras['Tidak Memuaskan'] = fuzz.trimf(ketersediaan_sarpras.universe, [0, 0, 75])
ketersediaan_sarpras['Cukup Memuaskan'] = fuzz.trimf(ketersediaan_sarpras.universe, [60, 75, 90])
ketersediaan_sarpras['Memuaskan'] = fuzz.trimf(ketersediaan_sarpras.universe, [75, 100, 100])

kepuasan_pelayanan['Tidak Memuaskan'] = fuzz.trimf(kepuasan_pelayanan.universe, [0, 0, 100])
kepuasan_pelayanan['Kurang Memuaskan'] = fuzz.trimf(kepuasan_pelayanan.universe, [50, 100, 150])
kepuasan_pelayanan['Cukup Memuaskan'] = fuzz.trimf(kepuasan_pelayanan.universe, [100, 175, 250])
kepuasan_pelayanan['Memuaskan'] = fuzz.trimf(kepuasan_pelayanan.universe, [175, 250, 325])
kepuasan_pelayanan['Sangat Memuaskan'] = fuzz.trimf(kepuasan_pelayanan.universe, [250, 400, 400])
```

Penjelasan: Langkah ketiga berfokus pada pembuatan fungsi keanggotaan menggunakan tipe kurva segitiga tunggal (fuzz.trimf) untuk memetakan deret angka-angka tersebut ke dalam himpunan linguistik, di mana variabel input dibagi menjadi kategori "Tidak Memuaskan", "Cukup Memuaskan", dan "Memuaskan", sementara variabel output kepuasan pelayanan dibagi menjadi lima tingkatan mulai dari "Tidak Memuaskan" hingga "Sangat Memuaskan" berdasarkan tiga titik potong koordinatnya.

```
kejelasan_informasi.view()
kejelasan_prasyarat.view()
kemampuan_petugas.view()
ketersediaan_sarpras.view()
kepuasan_pelayanan.view()
```

Penjelasan: Bagian keempat ini bertugas untuk memberikan visualisasi dari rancangan fungsi keanggotaan yang telah dibuat, di mana program memanggil fungsi .view() pada keempat variabel input beserta satu variabel output secara berurutan agar sketsa bentuk kurva mentah dari himpunan-himpunan fuzzy tersebut dapat digambar dan dipersiapkan sebelum nantinya digabungkan dengan hasil simulasi.

```
aturan1 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Tidak Memuaskan'] & kemampuan_petugas['Tidak Memuaskan'] & ketersediaan_sarpras['Tidak Memuaskan'], kepuasan_pelayanan['Kurang Memuaskan'])
aturan2 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Tidak Memuaskan'] & kemampuan_petugas['Tidak Memuaskan'] & ketersediaan_sarpras['Cukup Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
aturan3 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Tidak Memuaskan'] & kemampuan_petugas['Tidak Memuaskan'] & ketersediaan_sarpras['Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
aturan4 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Tidak Memuaskan'] & kemampuan_petugas['Cukup Memuaskan'] & ketersediaan_sarpras['Tidak Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
aturan5 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Tidak Memuaskan'] & kemampuan_petugas['Cukup Memuaskan'] & ketersediaan_sarpras['Cukup Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
aturan6 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Tidak Memuaskan'] & kemampuan_petugas['Cukup Memuaskan'] & ketersediaan_sarpras['Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
aturan7 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Tidak Memuaskan'] & kemampuan_petugas['Memuaskan'] & ketersediaan_sarpras['Tidak Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
aturan8 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Tidak Memuaskan'] & kemampuan_petugas['Memuaskan'] & ketersediaan_sarpras['Cukup Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
aturan9 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Tidak Memuaskan'] & kemampuan_petugas['Memuaskan'] & ketersediaan_sarpras['Memuaskan'], kepuasan_pelayanan['Memuaskan'])

aturan10 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Cukup Memuaskan'] & kemampuan_petugas['Tidak Memuaskan'] & ketersediaan_sarpras['Tidak Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
aturan11 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Cukup Memuaskan'] & kemampuan_petugas['Tidak Memuaskan'] & ketersediaan_sarpras['Cukup Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
aturan12 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Cukup Memuaskan'] & kemampuan_petugas['Tidak Memuaskan'] & ketersediaan_sarpras['Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
aturan13 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Cukup Memuaskan'] & kemampuan_petugas['Cukup Memuaskan'] & ketersediaan_sarpras['Tidak Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
aturan14 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Cukup Memuaskan'] & kemampuan_petugas['Cukup Memuaskan'] & ketersediaan_sarpras['Cukup Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
aturan15 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Cukup Memuaskan'] & kemampuan_petugas['Cukup Memuaskan'] & ketersediaan_sarpras['Memuaskan'], kepuasan_pelayanan['Memuaskan'])
aturan16 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Cukup Memuaskan'] & kemampuan_petugas['Memuaskan'] & ketersediaan_sarpras['Tidak Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
aturan17 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Cukup Memuaskan'] & kemampuan_petugas['Memuaskan'] & ketersediaan_sarpras['Cukup Memuaskan'], kepuasan_pelayanan['Memuaskan'])
aturan18 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Cukup Memuaskan'] & kemampuan_petugas['Memuaskan'] & ketersediaan_sarpras['Memuaskan'], kepuasan_pelayanan['Memuaskan'])

aturan19 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Memuaskan'] & kemampuan_petugas['Tidak Memuaskan'] & ketersediaan_sarpras['Tidak Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
aturan20 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Memuaskan'] & kemampuan_petugas['Tidak Memuaskan'] & ketersediaan_sarpras['Cukup Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
aturan21 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Memuaskan'] & kemampuan_petugas['Tidak Memuaskan'] & ketersediaan_sarpras['Memuaskan'], kepuasan_pelayanan['Memuaskan'])
aturan22 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Memuaskan'] & kemampuan_petugas['Cukup Memuaskan'] & ketersediaan_sarpras['Tidak Memuaskan'], kepuasan_pelayanan['Cukup Memuaskan'])
aturan23 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Memuaskan'] & kemampuan_petugas['Cukup Memuaskan'] & ketersediaan_sarpras['Cukup Memuaskan'], kepuasan_pelayanan['Memuaskan'])
aturan24 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Memuaskan'] & kemampuan_petugas['Cukup Memuaskan'] & ketersediaan_sarpras['Memuaskan'], kepuasan_pelayanan['Memuaskan'])
aturan25 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Memuaskan'] & kemampuan_petugas['Memuaskan'] & ketersediaan_sarpras['Tidak Memuaskan'], kepuasan_pelayanan['Memuaskan'])
aturan26 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Memuaskan'] & kemampuan_petugas['Memuaskan'] & ketersediaan_sarpras['Cukup Memuaskan'], kepuasan_pelayanan['Memuaskan'])
aturan27 = ctrl.Rule(kejelasan_informasi['Tidak Memuaskan'] & kejelasan_prasyarat['Memuaskan'] & kemampuan_petugas['Memuaskan'] & ketersediaan_sarpras['Memuaskan'], kepuasan_pelayanan['Sangat Memuaskan'])
```

Penjelasan: Pada tahap kelima, program merumuskan secara komprehensif 81 aturan logika bersyarat (IF-THEN) yang mencakup seluruh kemungkinan kombinasi dari keempat variabel input, di mana setiap deklarasi aturan menggunakan operator logika AND (dengan simbol &) untuk mengevaluasi

status kondisi masukan dan kemudian menautkannya secara langsung dengan salah satu kategori keputusan akhir pada variabel kepuasan pelayanan.

```
kepuasan_ctrl = ctrl.ControlSystem([
    aturan1, aturan2, aturan3, aturan4, aturan5, aturan6, aturan7, aturan8, aturan9,
    aturan10, aturan11, aturan12, aturan13, aturan14, aturan15, aturan16, aturan17, aturan18,
    aturan19, aturan20, aturan21, aturan22, aturan23, aturan24, aturan25, aturan26, aturan27,
    aturan28, aturan29, aturan30, aturan31, aturan32, aturan33, aturan34, aturan35, aturan36,
    aturan37, aturan38, aturan39, aturan40, aturan41, aturan42, aturan43, aturan44, aturan45,
    aturan46, aturan47, aturan48, aturan49, aturan50, aturan51, aturan52, aturan53, aturan54,
    aturan55, aturan56, aturan57, aturan58, aturan59, aturan60, aturan61, aturan62, aturan63,
    aturan64, aturan65, aturan66, aturan67, aturan68, aturan69, aturan70, aturan71, aturan72,
    aturan73, aturan74, aturan75, aturan76, aturan77, aturan78, aturan79, aturan80, aturan81
])
```

Penjelasan: Bagian keenam merupakan inti dari mesin inferensi untuk program ini, di mana seluruh 81 aturan yang telah dideklarasikan tadi dikumpulkan dan dibungkus ke dalam sebuah sistem kontrol (ctrl.ControlSystem), yang selanjutnya sistem tersebut dikonversi menjadi sebuah instrumen simulasi melalui fungsi ControlSystemSimulation agar lingkungan program benar-benar siap untuk menerima dan memproses data angka yang sesungguhnya.

```
simulasi_kepuasan = ctrl.ControlSystemSimulation(kepuasan_ctrl)

simulasi_kepuasan.input['kejelasan_informasi'] = 80
simulasi_kepuasan.input['kejelasan_prasyarat'] = 60
simulasi_kepuasan.input['kemampuan_petugas'] = 50
simulasi_kepuasan.input['ketersediaan_sarpras'] = 90
```

Penjelasan: Pada tahap ketujuh, nilai studi kasus yang riil dimasukkan ke dalam variabel instrumen simulasi, yakni kejelasan informasi bernilai 80, prasyarat 60, kemampuan petugas 50, dan ketersediaan sarpras 90, yang kemudian secara otomatis diproses oleh fungsi .compute() di dalam struktur blok pelindung try-except untuk menghitung nilai fuzzy-nya, mendefuzzifikasinya menjadi sebuah angka skor keluaran yang tegas, dan mencetaknya ke layar sekaligus mencegah program mati tiba-tiba jika terjadi *error* pada perhitungan.

```
  simulasi_kepuasan.compute()
  hasil = simulasi_kepuasan.output['kepuasan_pelayanan']
  print(f"Berdasarkan input yang diberikan, Nilai Kepuasan Pelayanan adalah: {hasil:.2f}")

  kepuasan_pelayanan.view(sim=simulasi_kepuasan)
  plt.show()

cept ValueError as e:
  print(f"Error dalam perhitungan: {e}")
```

Penjelasan: Terakhir, pada bagian kedelapan, program kembali memanggil fungsi .view() yang difokuskan pada variabel output kepuasan pelayanan dengan melampirkan parameter data dari hasil eksekusi komputasinya, lalu menutup siklus proses dengan memanggil perintah plt.show() untuk menampilkan rupa grafis secara nyata berupa kurva output yang sudah dilengkapi dengan area blok arsiran tebal serta sebuah garis lurus yang menandai letak spesifik dari angka skor kepuasan tersebut.